

Dijkstra Sequence Judgement

2026-05-25

Chapter 1

Introduction

1.1 Problem Description

Dijkstra's algorithm is a classic greedy algorithm used to solve the single-source shortest path problem in weighted graphs. During each step of the algorithm, it selects an unvisited vertex with the minimum distance estimate from the source and adds it to the set of vertices with finalized shortest paths.

Since multiple vertices may have the same minimum distance at a given step, there can be more than one valid sequence of vertices generated by the algorithm. This project aims to verify whether a given permutation of vertices constitutes a legal Dijkstra Sequence for a provided undirected connected graph.

1.2 Background

Conceived by Edsger W. Dijkstra in 1956, the algorithm maintains a shortest-path tree. The key operation is "Relaxation," where the discovery of a new vertex allows the algorithm to potentially find shorter paths to its neighbors. The validity of a sequence depends entirely on whether the greedy choice property is respected at every step.

Chapter 2

Algorithm Specification

2.1 Data Structures

- **Adjacency Matrix (`graph[][]`):** A 2D array of size 1001×1001 used to store edge weights of undirected graph. It provides $O(1)$ access time to check the distance between any two vertices.
- **Distance Array (`dist[]`):** An array used during verification to track the current shortest distance from the source to every other vertex.
- **Visited Array (`visited[]`):** A flag array to mark vertices that have been visited in the Dijkstra sequence.

2.2 Core Algorithm: Sequence Validity Check

For each given sequence `seq[0..Nv-1]` (with `seq[0]` as the source), simulate Dijkstra's algorithm and check the legality of the vertex order. The detailed steps are as follows:

1. Initialization

- Set all `visited[v] = 0` for $v = 1 \dots N_v$
- Set `dist[v] = INF` for $v = 1 \dots N_v$
- `dist[seq[0]] = 0`

2. Process each vertex in the sequence $u = \text{seq}[i]$ ($i = 0 \dots N_v - 1$):

- **Duplicate check:** If `visited[u]` isn't 0, the vertex appears more than once; the sequence is invalid, return 0.

- **Minimum-distance condition** (when $i > 0$): Scan all unvisited vertices v ; if there exists some v such that $\text{dist}[v] < \text{dist}[u]$, then a vertex with strictly smaller distance should have been selected instead of u ; the sequence is invalid, return 0.
- **Mark as visited**: $\text{visited}[u] = 1$
- **Relax adjacent edges**: Traverse all vertices v ($1 \leq v \leq N_v$). If $\text{!visited}[v] \ \&\& \ \text{graph}[u][v] \neq \text{INF}$, and if $\text{dist}[u] + \text{graph}[u][v] < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + \text{graph}[u][v]$.

3. If all vertices pass the checks, return 1 (the sequence is valid).

2.3 Pseudo-code

2.3.1 Dijkstra Sequence Judgement Function

```

FUNCTION is_dijkstra(sequence, graph, nv):
    SET dist[1...nv] TO INFINITY
    SET visited[1...nv] TO 0

    source = sequence[0]
    dist[source] = 0

    FOR i FROM 0 TO nv - 1:
        u = sequence[i]

        // 1. Verify Greedy Property
        min_distance = FIND_MIN_DIST(unvisited_nodes)
        IF dist[u] > min_distance:
            RETURN 0 // A closer node was skipped; invalid sequence

        visited[u] = 1

        // 2. Relaxation Process
        FOR EACH neighbor v of u:
            IF v is not visited AND graph[u][v] is not INFINITY:
                IF dist[u] + graph[u][v] < dist[v]:
                    dist[v] = dist[u] + graph[u][v]

    RETURN 1

```

Chapter 3

Testing Results

3.1 Test Cases

The following table summarizes the test cases, their purposes, expected outputs, actual outputs, and statuses.

Table 3.1: Test cases and results

ID	Purpose	Expected Result	Status
1	Original sample	Yes / Yes / Yes / No	Pass
2	Chain graph, different sources	Yes / No / Yes	Pass
3	Equal-distance branches, unreachable vertex appears early	Yes / Yes / No	Pass
4	Relaxation changes selection order	Yes / No	Pass
5	Source not 1, distant vertex chosen illegally early	Yes / Yes / No	Pass
6	Incorrect choice when a smaller distance exists	Yes / No	Pass
7	Large weight fails minimum-distance condition	Yes / No	Pass
8	Star graph with different sources	Yes / No	Pass
9	largest-scale star graph	Yes / No	Pass

3.2 Remarks

- **Coverage:** The test suite covers boundary cases (single vertex, chain), equal-distance ties, illegal vertex repetitions (not shown explicitly in this table but handled by the algorithm),

cases where relaxation changes the nearest vertex, and graphs with a source vertex other than 1.

- **Correctness:** All actual outputs match the expected results, confirming that the program correctly implements the Dijkstra sequence verification logic.
- **Performance:** Even for the maximum input size ($V = 1000$, $E = 10^5$, $K = 100$), the $O(K \cdot V^2)$ algorithm finishes within the time limit on standard judge platforms.
- **Limitations:** The current implementation uses an adjacency matrix; for very sparse graphs this is not memory-optimal, but for the given constraints it is acceptable. If the vertex limit were larger, an adjacency list would be more appropriate.

Chapter 4

Analysis and Comments

4.1 Time Complexity Analysis

- Initialization of distance and visited arrays: $O(V)$
- For each of the V vertices in the sequence:
 - Checking the minimum-distance condition requires scanning all unvisited vertices: $O(V)$
 - Relaxation using the adjacency matrix also scans all V vertices: $O(V)$
- Total time per sequence: $O(V^2)$
- For K queries: $O(K \cdot V^2)$

In the worst case ($V \leq 1000$, $K \leq 100$), the number of operations is about 10^8 , which is acceptable in C within typical time limits.

4.2 Space Complexity Analysis

- Adjacency matrix: $O(V^2)$, storing V^2 integers; worst case about 4 MB ($1001^2 \approx 1,002,001$ ints), well within memory limits.
- Auxiliary arrays `dist` and `visited`: $O(V)$

4.3 Possible Improvements

1. **Adjacency list for faster relaxation:** Although checking the minimum distance still requires scanning V vertices ($O(V^2)$), relaxation can be limited to actual neighbors using an

adjacency list, reducing the constant factor. This is especially effective for sparse graphs ($E \ll V^2$). Using an adjacency list would reduce the relaxation step to $O(E)$, but the scan for finding the minimum distance candidate still requires $O(V^2)$ comparisons. A min-heap could optimize verification further, but extracting a specific vertex on demand complicates the implementation.

2. **Input Validation:** The current implementation assumes valid input (connected graph, valid vertex numbers). Additional input validation could be added.

Appendix: Source Code (in C)

```
dijkstra.c

1 //check if the given sequence is a valid Dijkstra's algorithm
  order
2 int is_dijkstra(int seq[]){
3     int dist[MAX_VEX];           //the shortest distance from the
      source vertex to vertex i
4     int visited[MAX_VEX];       //visited sign of vertex i
5
6     //initialize dist and visited arrays
7     for(int i = 1; i <= nv; i++){
8         dist[i] = INF;           //initialize the shortest
      distance to all vertices to infinity
9         visited[i] = 0;         //initialize all vertices as
      unvisited
10    }
11    dist[seq[1]] = 0;            //set the first vertex in the
      sequence as the source vertex
12
13    for(int i = 1; i <= nv; i++){
14        if(i){
15            for (int v = 1; v <= nv; v++) {
16                if (!visited[v] && dist[v] < dist[seq[i]]) {
17                    //check if the current vertex has a
      shorter distance than the one in the sequence
18                    return 0;
19                }
20            }
21        }
22    }
```

```

21
22     visited[seq[i]] = 1;           //mark the current
        vertex as visited
23
24     for(int v = 1; v <= nv; v++){
25         if(!visited[v] && graph[seq[i]][v] != INF){
                //check if there is an edge between
        the current vertex and vertex v
26         if(dist[seq[i]] + graph[seq[i]][v] < dist[v]){
27             dist[v] = dist[seq[i]] + graph[seq[i]][v];
                //update the shortest distance to
        vertex v
28         }
29     }
30 }
31 }
32 return 1;
33 }

```

graph.c

```

1 int graph[MAX_VEX][MAX_VEX];    //use adjacency matrix to store
        the graph
2 int nv, ne;                     //number of vertices and edges
3
4 //initialize
5 void init(int nv){
6     for(int i = 1; i <= nv; i++){
7         for(int j = 1; j <= nv; j++){
8             if(i == j){
9                 graph[i][j] = 0;           //the distance from a
        vertex to itself is 0
10            } else {
11                graph[i][j] = INF;         //initialize the
        distance between different vertices to
        infinity

```

```

12         }
13     }
14 }
15 }
16
17 //add an weighed edge to the graph
18 void add_edge(int u, int v, int w){
19     //the graph is undirected, we need to update both graph[u][v
20         ] and graph[v][u]
21     graph[u][v] = w;
22     graph[v][u] = w;
23 }

```

```

main.c

1 #include<stdio.h>
2 #include <limits.h>
3
4 #define MAX_VEX 1001
5 #define INF INT_MAX
6
7 #include "graph.c"
8 #include "dijkstra.c"
9
10 int main(){
11     int k;
12
13     freopen("input.txt", "r", stdin);
14
15     //input until EOF, convenient for multiple test cases
16     while(scanf("%d %d",&nv,&ne) == 2){ //input
17         the number of vertices and edges
18         init(nv); //initialize the
19         graph
20
21         //input the edges and their weights

```

```

20     for(int i = 0; i < ne; i++){
21         int u, v, w;
22         scanf("%d %d %d",&u,&v,&w);
23         if(w < graph[u][v]){
24             add_edge(u, v, w);           //if there are
                                           multiple edges between the same vertices, we
                                           only keep the one with the smallest weight
25         }
26     }
27
28     scanf("%d",&k);           //the number of sequences to
                               check
29     int seq[MAX_VEX];         //the array to store the
                               sequence of vertices to check
30     for(int i = 0; i < k; i++){
31         for(int j = 1; j <= nv; j++){
32             scanf("%d",&seq[j]);       //input the
                                           sequence of vertices, starting from index 1
33         }
34
35         //check if the sequence is a valid Dijkstra's
           algorithm order and print the result
36         if(is_dijkstra(seq)){
37             printf("Yes\n");
38         } else {
39             printf("No\n");
40         }
41     }
42 }
43 return 0;
44 }

```

Declaration

I hereby declare that all the work done in this project titled “Dijkstra’s Algorithm Judgement” is of my independent effort.